Programming Concepts of Basic Java

Concise Study Notes for OAVS Computer Teacher Competitive Examination — Revision-Focused

Contents

1. Identifiers and Keywords
2. Data Types in Java
3. Java Coding Conventions
4. Expressions in Java
5. Control Structures, Decision Making Statements
6. Arrays and its Methods

1 Identifiers and Keywords

Definition

Identifiers are names given by the programmer to identify program elements such as variables, methods, classes, interfaces, packages, and labels. **Keywords** are reserved words that have a predefined meaning in the Java language and cannot be used as identifiers.

Rules for Valid Identifiers

- Can contain letters (A–Z, a–z), digits (0–9), underscore (`_`), and dollar sign (`$`).
- Must **not start with a digit**; it may start with a letter, `_`, or `$`.
- Cannot contain spaces or special symbols like `@ # % ^ & * ( ) - + = ! ~`.
- Cannot be a Java **reserved keyword**.
- Java identifiers are **case-sensitive** (`Total` and `total` are different).
- No length limit is specified by the language.

Valid Identifiers	Invalid Identifiers	Reason (Invalid)
`total` , `_count` , `$price` , `num1` , `student_Name`	`1num`	Starts with a digit
`MyClass` , `a1b2`	`my-name`	Contains hyphen (-)
`_temp`	`class`	Reserved keyword
`value$`	`my name`	Contains a space

Java Keywords

Java has **53 reserved keywords** as per Java SE (including `const` and `goto`, which are reserved but unused). `true` , `false` , and `null` are technically **literals**, not keywords, though often grouped with them. `var` (Java 10+) is a **reserved type name**, not a full keyword — it can still be used as an identifier in limited contexts.

Category	Keywords
Data type keywords	`byte, short, int, long, float, double, char, boolean, void`
Access modifiers	`public, private, protected`
Class/Object related	`class, interface, extends, implements, new, this, super, package, import`

Control flow	`if, else, switch, case, default, for, while, do, break, continue, return`
Exception handling	`try, catch, finally, throw, throws`
Modifiers	`static, final, abstract, synchronized, transient, volatile, native, strictfp`
Others	`instanceof, enum, assert, const*, goto*` (*reserved, unused)

Updated Count — Latest Java Versions:

As of the latest versions of Java, there are **68 reserved words** in total. This breakdown includes **51 standard keywords**, **2 keywords that are reserved but currently unused** (`const` and `goto`), and **15 contextual keywords** (introduced in newer releases like Java 9 through Java 21).

Exam-oriented facts:

- Total keyword count commonly asked as **50 (as per old spec) / 53 (with enum, assert, const, goto)** — OAVS papers usually expect **53** (51 in use + 2 reserved-unused). Newer syllabi may reference the updated count of **68** (see note above).
- `enum` was added in Java 5; `assert` was added in Java 1.4.
- Keywords are always written in **lowercase**.
- `true` , `false` , `null` are Boolean/null **literals** — a frequently confused MCQ point.

⚠ **Common mistakes:**

- Thinking identifiers can start with a digit — they cannot.
- Treating `String` as a keyword — it is a **class name**, not a keyword.
- Forgetting Java is case-sensitive, so `Class` (capital C) is a valid identifier while `class` is not.

Summary: Identifiers name program elements and must follow letter/digit/_/$ rules without starting with a digit; keywords are fixed reserved words (53 total) that cannot be reused as identifiers.

② Data Types in Java

Definition

A **data type** specifies the type of value a variable can hold and the set of operations that can be performed on it. Java is a **strongly-typed** language — every variable must be declared with a data type.

Classification

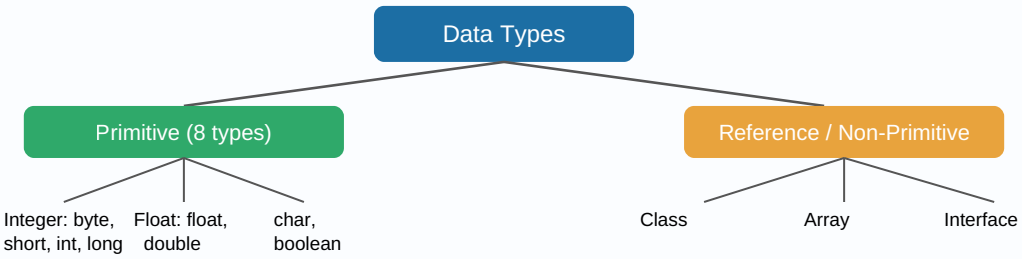


Fig 1: Classification of Java Data Types

Primitive Data Types — Sizes and Ranges

Type	Size	Default Value	Range
`byte`	1 byte (8 bit)	0	-128 to 127
`short`	2 byte (16 bit)	0	-32,768 to 32,767
`int`	4 byte (32 bit)	0	-2,147,483,648 to 2,147,483,647
`long`	8 byte (64 bit)	0L	-9.2×10^{18} to 9.2×10^{18}
`float`	4 byte (32 bit)	0.0f	$\sim\pm3.4\times10^{38}$ (6–7 decimal digits precision)
`double`	8 byte (64 bit)	0.0d	$\sim\pm1.8\times10^{308}$ (15 decimal digits precision)
`char`	2 byte (16 bit)	'\u0000'	0 to 65,535 (unsigned, Unicode)
`boolean`	~1 bit (JVM-dependent, not precisely defined)	false	true / false only

Reference (Non-Primitive) Data Types

- **Class type** — objects created from a class, e.g. `String s = "hello";`
- **Array type** — collection of similar elements, e.g. `int[] arr;`
- **Interface type** — reference to any implementing class object.

A reference type stores the **address (reference)** of the object in memory, not the value itself, and its default value is `null`.

Example

```
public class DataTypeDemo {
    public static void main(String[] args) {
        int age = 25;
        double salary = 45000.50;
        char grade = 'A';
        boolean isPassed = true;
        System.out.println(age + " " + salary + " " + grade + " " + isPassed);
    }
}
```

Output: `25 45000.5 A true`

Reference Type vs Pointer:

A Java reference is similar to a pointer — it stores an object's address rather than its data. But unlike C/C++ pointers, Java hides the actual address, disallows pointer arithmetic, and manages memory automatically via the Garbage Collector. Hence Java is said to have **references, not true pointers**.

Exam-oriented facts:

- Java has exactly **8 primitive types**: byte, short, int, long, float, double, char, boolean.
- `char` is **unsigned** and unique — it stores Unicode characters, size 2 bytes.
- Default numeric literal type is `int`; default floating literal type is `double` (use suffix `L` for long, `f` / `F` for float).
- `String` is **not** a primitive type — it is a class in `java.lang`.

⚠ **Common mistakes:**

- Writing `float f = 3.14;` without `f` suffix causes a compile error (3.14 is treated as `double`).
- Confusing `char` size (2 bytes) with C/C++ `char` (1 byte).
- Assuming `boolean` has a fixed memory size — JVM implementation dependent.

Summary: Java has 8 primitive types (numeric, char, boolean) with fixed sizes/ranges, and reference types (class, array, interface) that store object addresses with default value `null`.

③ Java Coding Conventions

Definition

Coding conventions are standard guidelines recommended by Oracle/Sun Microsystems for writing readable, consistent, and maintainable Java code. They are not enforced by the compiler but are important for professional practice and are frequently tested for naming-rule identification.

Naming Conventions

Element	Convention	Example
Class / Interface	Nouns, PascalCase (UpperCamelCase)	`StudentRecord`, `Runnable`
Method	Verbs, camelCase	`calculateTotal()`, `getName()`
Variable	camelCase, meaningful nouns	`studentName`, `totalMarks`
Constant	`static final`, ALL_CAPS with underscore	`MAX_VALUE`, `PI`
Package	All lowercase, reverse domain style	`com.oavs.app`

Formatting Conventions

- Use **4 spaces** per indentation level (no tabs preferred in Sun's guide).
- Opening brace `{` at the end of the line (K&R style), not on a new line.
- One statement per line; one declaration per line (avoid `int a, b;` style where possible).
- File name must exactly match the **public class name**, including case, with `.java` extension.
- Only **one public class** is allowed per source file.
- Meaningful, self-descriptive names should be used instead of single letters (except loop counters like `i, j, k`).

Example (Good vs Poor Convention)

```
// Good convention
public class Employee {
    private int employeeId;
    public static final int MAX_AGE = 60;

    public int getEmployeeId() {
        return employeeId;
    }
}
```

```
// Poor convention
public class employee {
    private int EMP_id;
    public static final int maxage = 60;

    public int Getid(){
    return EMP_id;}
}
```

Exam-oriented facts:

- Java is **case-sensitive**, so convention violations don't stop compilation, but file-name mismatch with the public class **does** cause a compile error.

- Conventions were originally documented by **Sun Microsystems** ("Code Conventions for the Java Programming Language").
- Constants must be declared `static final` and named in UPPER_CASE.

⚠ **Common mistakes:**

- Naming a class starting with a lowercase letter (should be PascalCase).
- Believing conventions are compiler-enforced rules — they are only recommended style practices (except the public-class/filename rule, which IS enforced).

Summary: Classes use PascalCase, methods/variables use camelCase, constants use UPPER_CASE, packages use lowercase; the public class name must match the file name — a rule the compiler actually enforces.

4 Expressions in Java

Definition

An **expression** is a combination of **operands** (variables, literals, method calls) and **operators** that evaluates to a single value. Example: `a + b * c` is an expression.

Operands and Operators

Operand — the data on which an operator acts (e.g., variables, constants). **Operator** — a symbol that performs an operation on operands.

Category	Operators	Example
Arithmetic	`+ - * / %`	`a + b`
Unary	`+ - ++ -- !`	`-a`, `a++`
Relational	`== != > < >= <=`	`a > b`
Logical	`&& || !`	`a>0 && b>0`
Assignment	`= += -= *= /= %=`	`a += 5`
Bitwise	`& | ^ ~ << >> >>>`	`a & b`
Conditional (Ternary)	`? :`	`a>b ? a : b`
instanceof	`instanceof`	`obj instanceof String`

Precedence and Associativity

When an expression has multiple operators, **precedence** decides which operator is applied first; **associativity** decides the direction of evaluation (left-to-right or right-to-left) among operators of equal precedence.

Precedence (High→Low)	Operator(s)	Associativity	Example
1	`() [] .` (postfix `++ --`)	Left to Right	`arr[0]`, `obj.method()`, `x++`
2	Unary `++ -- + - ! ~`	Right to Left	`++x`, `-a`, `!flag`
3	`* / %`	Left to Right	`6 * 2` → 12
4	`+ -`	Left to Right	`10 + 5 - 3` → 12
5	`<< >> >>>`	Left to Right	`4 << 1` → 8
6	`< <= > >= instanceof`	Left to Right	`a > b`, `obj instanceof String`

7	`== !=`	Left to Right	`a == b` → true/false
8	`&`	Left to Right	`5 & 3` → 1
9	`^`	Left to Right	`5 ^ 3` → 6
10	`|`	Left to Right	`5 | 3` → 7
11	`&&`	Left to Right	`a>0 && b>0`
12	`||`	Left to Right	`a>0 || b>0`
13	`?:` (ternary)	Right to Left	`a>b ? a : b`
14	`= += -= *= /= %=`	Right to Left	`a = b = 5`

Evaluation Example

```
int result = 10 + 6 * 2 - 4 / 2;
```

Step 1: * and / (left to right, higher precedence)

```
6 * 2 = 12 and 4 / 2 = 2
```

Step 2: 10 + 12 - 2 → + and - (left to right)

```
10 + 12 = 22 then 22 - 2 = 20
```

```
result = 20
```

Fig 2: Step-by-step precedence evaluation

```
public class ExprDemo {
    public static void main(String[] args) {
        int result = 10 + 6 * 2 - 4 / 2;
        System.out.println("Result = " + result);
    }
}
```

Output: `Result = 20`

Exam-oriented facts:

- Multiplication/Division/Modulus have **higher precedence** than Addition/Subtraction.
- Assignment operators are evaluated **right to left** — important for chained assignment: `a = b = c = 5;`
- Ternary operator `?:` is the **only** ternary (3-operand) operator in Java.
- Parentheses `()` always have the highest precedence and can override default order.

⚠ **Common mistakes:**

- Confusing `=` (assignment) with `==` (equality) — a top exam trap.
- Assuming `&&` and `||` have same precedence — `&&` binds tighter than `||`.
- Ignoring integer division truncation: `7 / 2` gives `3`, not `3.5`.

Summary: Expressions combine operands and operators; operator precedence (with parentheses highest) and left-to-right/right-to-left associativity determine the evaluation order and final result.

5 Control Structures, Decision Making Statements

Definition

Control structures alter the normal sequential (top-to-bottom) flow of program execution. **Decision-making statements** execute different blocks of code depending on whether a condition evaluates to `true` or `false`.

Types of Control Flow

Type	Statements
Selection (Decision)	`if`, `if-else`, `else-if` ladder, nested `if`, `switch`
Iteration (Looping)	`for`, `while`, `do-while` not in scope
Jump	`break`, `continue`, `return` not in scope

Note: As per the syllabus, this section focuses on decision-making (selection) statements only.

1. Simple if Statement

```
if (condition) {
    // executes only if condition is true
}
```

2. if-else Statement

```
if (condition) {
    // true block
} else {
    // false block
}
```

3. else-if Ladder

```
if (condition1) {
    // block 1
} else if (condition2) {
    // block 2
} else {
    // default block
}
```

4. Nested if

```
if (condition1) {
    if (condition2) {
        // executes if both true
    }
}
```

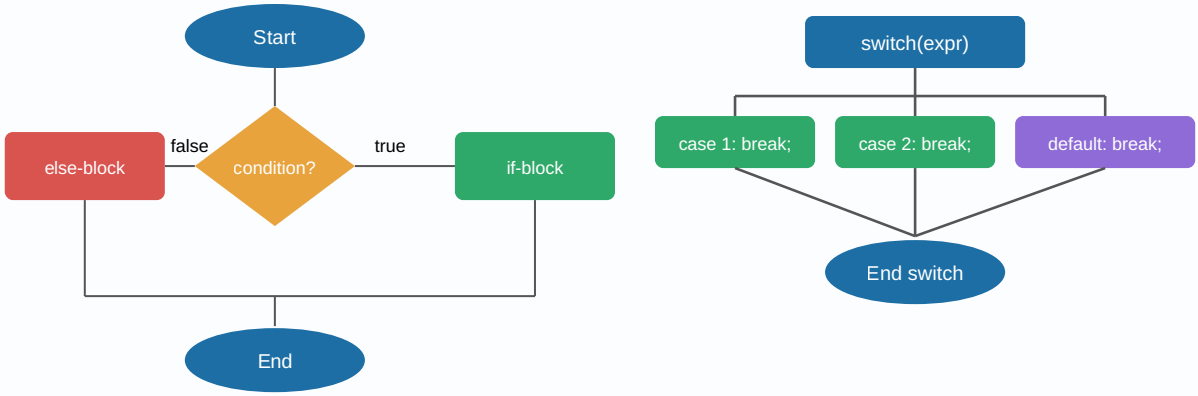


Fig 3: if-else flowchart (left) and switch-case structure flow (right)

5. switch Statement

```
switch (expression) {
    case value1:
        // statements
        break;
    case value2:
        // statements
        break;
    default:
        // statements
}
```

switch works with	switch does NOT work with
`byte, short, int, char`, their wrapper classes, `String` (Java 7+), `enum`	`long, float, double, boolean`

Example: Grade Checker (if-else-if)

```
public class GradeCheck {
    public static void main(String[] args) {
        int marks = 78;
        if (marks >= 90) {
            System.out.println("Grade A");
        } else if (marks >= 75) {
            System.out.println("Grade B");
        } else if (marks >= 50) {
            System.out.println("Grade C");
        } else {
            System.out.println("Fail");
        }
    }
}
```

Output: `Grade B`

Exam-oriented facts:

- Without `break`, switch execution **falls through** to the next case automatically.
- `switch` expression must evaluate to `byte, short, int, char, String,` or `enum` — never `boolean`, `long`, `float`, or `double`.
- The `default` case can appear anywhere in a switch block, not only at the end.
- Only one `else` is allowed per `if`, but multiple `else if` are allowed.

⚠ **Common mistakes:**

- Forgetting `break;` in switch statements, causing unintended fall-through.
- Using `=` instead of `==` inside an `if` condition.
- Misplacing `else` so it binds to the wrong (nearest) `if` in nested-if code — "dangling else" problem.

Summary: Decision-making statements (`if` , `if-else` , `else-if` , nested `if` , `switch`) direct program flow based on conditions; `switch` supports only specific data types and needs `break` to avoid fall-through.

6 Arrays and its Methods

Definition

An **array** is a fixed-size, index-based data structure that stores multiple values of the **same data type** in contiguous memory locations. An array is itself a **reference (object) type** in Java.

Declaration, Creation, Initialization

```
// Declaration
int[] marks;          // preferred style
int marks[];          // also valid (C-style)

// Creation
marks = new int[5];   // creates array of 5 ints, default value 0

// Combined declaration + creation
int[] marks = new int[5];

// Declaration + Initialization (values known)
int[] marks = {90, 85, 76, 60, 99};
```

Indexing and Access

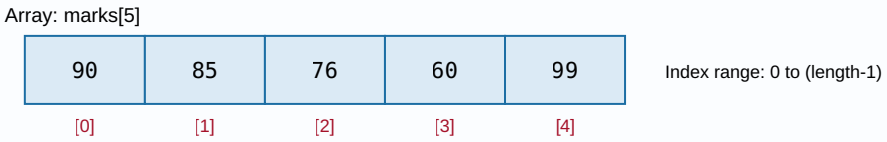


Fig 4: Array memory / index representation (marks.length = 5)

```
System.out.println(marks[0]);   // 90 (first element)
System.out.println(marks[4]);   // 99 (last element)
System.out.println(marks.length); // 5 (array length, no parentheses)
```

Multidimensional Arrays

Java supports arrays of arrays (2D, 3D, etc.). A 2D array is commonly visualized as a matrix (rows × columns).

```
int[][] matrix = new int[2][3];       // 2 rows, 3 columns
int[][] matrix2 = {{1,2,3}, {4,5,6}}; // direct initialization
System.out.println(matrix2[1][2]);    // row 1, column 2 → 6
```

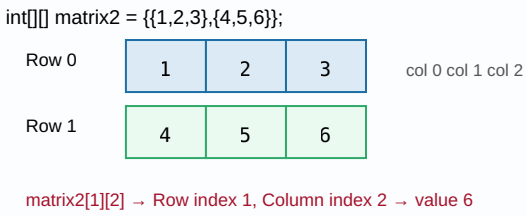


Fig 5: Multidimensional (2D) array representation

Traversing an Array

Traversing means visiting/accessing every element of the array one by one, in order — typically to read, print, process, or check each value. A loop (usually `for`) is used to walk through the array from index `0` to `length - 1`.

```
int[] marks = {90, 85, 76, 60, 99};
for (int i = 0; i < marks.length; i++) {
    System.out.println("marks[" + i + "] = " + marks[i]);
}
```

Output: `marks[0] = 90  marks[1] = 85  marks[2] = 76  marks[3] = 60  marks[4] = 99`

Important Array Methods / Utilities (java.util.Arrays)

Method	Purpose	Example
`length` (field, not method)	Returns number of elements	`marks.length`
`Arrays.sort(arr)`	Sorts array in ascending order	`Arrays.sort(marks);`
`Arrays.toString(arr)`	Converts array to printable String	`Arrays.toString(marks)`
`Arrays.equals(a1, a2)`	Checks if two arrays have identical elements	`Arrays.equals(a,b)`
`Arrays.fill(arr, val)`	Fills all elements with given value	`Arrays.fill(arr, 0);`
`Arrays.copyOf(arr, n)`	Copies array to a new array of size n	`Arrays.copyOf(arr,10)`
`Arrays.binarySearch(arr, key)`	Searches key in a sorted array	`Arrays.binarySearch(arr, 76)`

```
import java.util.Arrays;
int[] marks = {90, 85, 76, 60, 99};

System.out.println(marks);                  // without toString()
Arrays.sort(marks);
System.out.println(Arrays.toString(marks));  // with toString()
```

Output: `[I@1b6d3586  (without toString – memory reference)`
`[60, 76, 85, 90, 99]  (with toString – readable content)`

Exam-oriented facts:

- Array **index always starts at 0**; valid indices go up to `length - 1`.
- `length` is a **field** (no parentheses) for arrays, while `length()` is a **method** for Strings — a classic exam trap.
- Array size is **fixed** at creation time and cannot be changed later.
- Accessing an invalid index throws `ArrayIndexOutOfBoundsException` at runtime.

- Arrays are objects in Java; default values on creation follow the primitive type's default (0, 0.0, false, '\u0000', or null for reference-type arrays).

⚠ **Common mistakes:**

- Using `marks.length()` instead of `marks.length` — causes compile error.
- Off-by-one errors: looping with `i <= marks.length` instead of `i < marks.length`.
- Assuming array size can grow — must create a new array (e.g., via `Arrays.copyOf`) to "resize".
- Confusing `int[][] arr = new int[3][]` (jagged array, rows of unequal length allowed) with a fixed rectangular 2D array.

⚡ **Summary:** Arrays store fixed-size, same-type elements accessed via zero-based indices; multidimensional arrays extend this to rows/columns, and the `java.util.Arrays` class provides utility methods like `sort()`, `toString()`, and `binarySearch()`.

Quick Revision — Key Facts & Likely Exam Points

Topic	Must-Remember Points
Identifiers & Keywords	No digit at start; case-sensitive; 53 reserved keywords (incl. unused `const`, `goto`); `true/false/null` are literals, not keywords.
Data Types	8 primitive types; `char` = 2 bytes, unsigned Unicode; default numeric literal = `int`, default decimal literal = `double`; reference types default to `null`.
Coding Conventions	Class → PascalCase; method/variable → camelCase; constant → UPPER_CASE; file name must match public class name (compiler-enforced).
Expressions	Precedence order: parentheses > unary > `* / %` > `+ -` > relational > `==/!=` > `&&` > `||` > `?:` > assignment; assignment associates right-to-left.
Control Structures	`switch` supports byte/short/int/char/String/enum only (not boolean/long/float/double); missing `break` causes fall-through.
Arrays	Index starts at 0; `length` is a field (no parentheses); fixed size; `ArrayIndexOutOfBoundsException` on invalid index; use `java.util.Arrays` for `sort()`, `toString()`, `binarySearch()`.

High-yield one-liners for MCQs:

- Total Java keywords = **53** (51 actively used + 2 reserved-unused: `const`, `goto`).
- `String.length()` vs `array.length` — method vs field, most repeated trap.
- Integer division truncates decimals: `7/2 = 3`.
- Ternary `?:` is Java's only three-operand operator.
- Array elements share the same data type; default index range is `0` to `length-1`.
- `switch` cannot use `boolean`, `long`, `float`, or `double` as its expression type.